

## UNIT-III

### Short Answer Type Questions

#### 1. What is the purpose of the bisection method in numerical computation?

The primary purpose of the bisection method is to find the root of a continuous function, which is the value of  $x$  for which  $f(x) = 0$ . It's a reliable method that systematically narrows down an interval that is known to contain a root.

#### 2. State one advantage of the Newton-Raphson method.

The main advantage of the Newton-Raphson method is its fast rate of convergence. It typically converges quadratically, meaning the number of correct decimal places roughly doubles with each iteration, making it much faster than methods like bisection.

#### 3. What is a matrix in linear algebra?

A matrix is a rectangular array or grid of numbers, symbols, or expressions arranged in rows and columns. It is a fundamental tool in linear algebra used to represent data, systems of linear equations, and linear transformations.

#### 4. Why does the bisection method require opposite signs at the interval ends?

The bisection method requires  $f(a)$  and  $f(b)$  to have opposite signs because this condition, according to the **Intermediate Value Theorem**, guarantees that the continuous function  $f(x)$  must cross the  $x$ -axis at least once within the interval  $[a, b]$ . This ensures a root exists in the interval to be found.

#### 5. Explain the term numerical integration in brief.

Numerical integration is the process of approximating the value of a definite integral (the area under a curve). It is used when it is difficult or impossible to find the exact value of the integral analytically (using standard integration rules). **Numerical integration Methods Examples** :Trapezoidal Rule,simpson's rule.

## 6. What is the role of eigenvalues in linear algebra?

Eigenvalues are special scalars associated with a linear system of equations.

- ✧ They help us understand how a matrix behaves.
- ✧ They are used to simplify big matrix problems.
- ✧ They tell us if a system (like in physics or engineering) is stable or not.
- ✧ They are crucial for understanding matrix transformations
- ✧ In data science, they help us find the important patterns in data like in PCA(Principal Component Analysis) and facial recognition.

## 7. Apply the Newton-Raphson method to find the root of $x^2 - 4 = 0$ with initial guess $x_0=1$ .

*GIVEN:*

$$\text{Initial Guess : } x_0 = 1 \qquad f(x) = x^2 - 4 \qquad f'(x) = 2x$$

$$\text{Next Guess: } x_1 = x_0 - f(x_0)/f'(x_0) \qquad \text{Tolerance } tol=0.1 \text{ (assuming)}$$

```
f = lambda x: x**2 - 4
f_prime = lambda x: 2*x
def my_newton(f, df, x0, tol):
    if abs(f(x0)) < tol:
        return x0
    else:
        return my_newton(f, df, x0 - f(x0)/df(x0), tol)
estimate = my_newton(f, f_prime, 1, 0.1)
print("estimate =", estimate)
OUTPUT:
estimate = 2.000609756097561
```

## 8. Use the trapezoidal rule to integrate $f(x) = x^2$ from 0 to 2 using 4 intervals.

*GIVEN:*

Start:  $a=0$

End :  $b=2$

Width :  $h=b-a/n-1$

Intervals: 4intervals means 5 grid points therefore,  $n=5$

```
import numpy as np
a = 0
b = 2
n = 5
h = (b - a) / (n-1)
x = np.linspace(a, b, n) //Equally divides the intervals.
f = x**2
I_trap = (h/2)*(f[0] + 2 * sum(f[1:n-1])) + f[n-1])
print(I_trap)
```

OUTPUT:  
2.75

## 9. Why might Newton-Raphson fail to converge?

The Newton-Raphson method is a powerful root-finding algorithm, but it can fail to converge for several reasons.

**Bad starting point:** If you start too far from the root, it may go in the wrong direction.

**Derivative becomes zero:** You'll end up dividing by zero, which is not allowed.

**It keeps jumping around:** Sometimes it doesn't settle at the root and keeps bouncing (oscillates or diverges).

**Root is flat (like a double root):**The method becomes very slow because it takes tiny steps.

## Long Answer Type Questions

### 1. Explain the Bisection Method with its algorithm, flowchart, advantages, and disadvantages.

The Bisection Method is a simple and reliable (though sometimes slow) way to find the root of a continuous function. It works on the principle of the Intermediate Value Theorem, which states that if a continuous function  $f(x)$  has values of opposite sign at two points,  $a$  and  $b$ , then it must have at least one root between them.

#### Algorithm:

1. **Start:** Choose a continuous function  $f(x)$  and an initial interval  $[a, b]$ .
2. **Check:** Verify that  $f(a)$  and  $f(b)$  have opposite signs (i.e.,  $f(a) * f(b) < 0$ ). If not, the method cannot proceed.
3. **Loop:** Repeat the following steps until the interval  $(b-a)$  is smaller than a predefined tolerance ( $tol$ ).
  - a) **Calculate Midpoint:** Find the midpoint of the interval:  $c = (a + b) / 2$ .
  - b) **Check for Root:** If  $f(c)$  is very close to 0,  $c$  is the root.
  - c) **Update Interval:** If  $f(a) * f(c) < 0$ , the root lies in the first half. Set  $b = c$ .  
Else, the root lies in the second half. Set  $a = c$ .
4. **End:** The final midpoint  $c$  is the approximation of the root.

#### Advantages:

**Reliability:** It is guaranteed to converge to a root if the initial interval is chosen correctly.

**Simplicity:** The algorithm is easy to understand and implement.

**Predictable Error:** The error bound is known and decreases by a factor of 2 with each iteration.

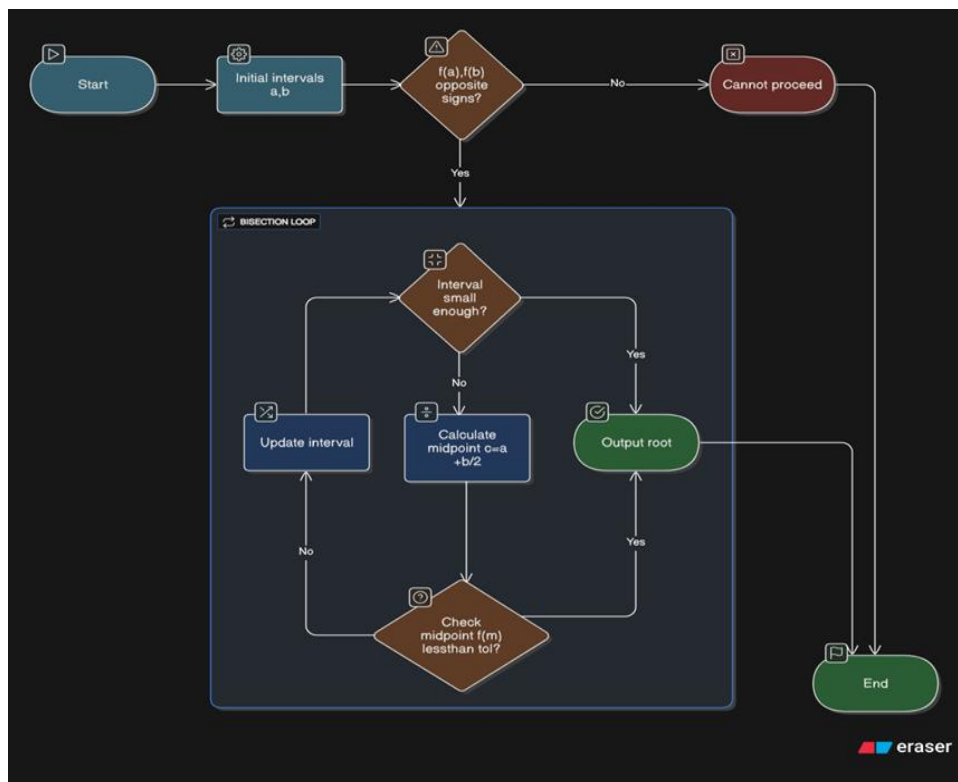
### Disadvantages:

**Slow Convergence:** The rate of convergence is linear, which is very slow compared to other methods like Newton-Raphson.

**Multiple Roots:** If there are multiple roots in the initial interval, it will only find one of them.

**Initial Guess:** Requires two initial guesses that bracket the root, which may not be easy to find for all functions.

### FLOWCHART:



## 2. Compare the Bisection Method and Newton-Raphson Method in terms of convergence and applicability.

The Bisection and Newton-Raphson methods are two popular but fundamentally

| Feature                          | Bisection Method                                                                                                                           | Newton-Raphson Method                                                                                                                    |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Convergence Rate</b>          | <b>Linear (Slow):</b> The error is halved at each step. Convergence is predictable and steady but takes many iterations for high accuracy. | <b>Quadratic (Very Fast):</b> The number of correct digits roughly doubles with each iteration, provided the initial guess is good.      |
| <b>Initial Guess(es)</b>         | Requires <b>two</b> initial guesses (a and b) that must bracket the root ( $f(a) * f(b) < 0$ ).                                            | Requires only <b>one</b> initial guess ( $x_0$ ) which should be reasonably close to the actual root.                                    |
| <b>Function Requirements</b>     | The function must be <b>continuous</b> on the interval $[a, b]$ .                                                                          | The function and its <b>first derivative</b> must be continuous. The derivative at the root should not be zero.                          |
| <b>Reliability/Robustness</b>    | <b>Highly reliable.</b> It is guaranteed to find a root if the initial conditions are met.                                                 | <b>Less reliable.</b> It can fail to converge (diverge or oscillate) if the initial guess is poor or if the derivative is close to zero. |
| <b>Implementation Complexity</b> | <b>Simple.</b> It does not require the calculation of the function's derivative.                                                           | <b>More complex.</b> It requires computing the derivative of the function, which can be difficult or computationally expensive.          |

### Applicability :

#### Choose Bisection when:

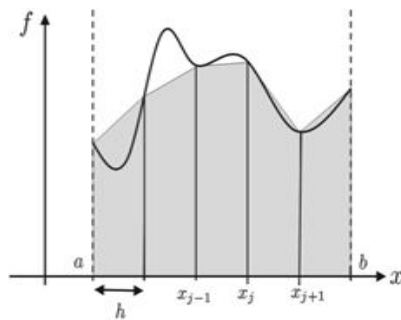
- You need a **guaranteed and reliable** result.
- The function's derivative is difficult or impossible to compute.
- Speed is not the highest priority.

#### Choose Newton-Raphson when:

- **Speed is critical**, and you need high precision quickly.
- You can provide a good initial guess close to the root.
- The function's derivative is known and easy to calculate.

### 3. Write a Python program to implement the Trapezoidal Rule and demonstrate its use.

The Trapezoidal Rule approximates a definite integral by dividing the area under the function's curve into a series of trapezoids and summing their areas.



**Function:**  $f(x) = \sin(x)$

**Interval:**  $a = 0, b = \pi$

**Exact Value:**  $\int_0^{\pi} \sin(x) dx = 2$  (Demonstrate it to a known function  $\sin(x)$  with known exact value)

### Python Implementation:

```
import numpy as np
a = 0 # Define interval and number of points
b = np.pi
n = 11 # number of points (10 intervals)
h = (b - a) / (n - 1) # width of each interval
x = np.linspace(a, b, n) # Create x values and compute f(x)
f = np.sin(x)
I_trap = (h / 2) * (f[0] + 2 * sum(f[1:n-1]) + f[n-1]) # Trapezoidal rule
implementation
exact = 2
err_trap = exact - I_trap # Compute error
print("Approximate integral:", I_trap)
print("Error:", err_trap)
OUTPUT:
Approximate integral: 1.9835235375094546
Error: 0.01647646249054535
```

### 4. Implement the Bisection Method in Python to find a root of $f(x) = \cos(x) - x$ , explaining each step.

```
import numpy as np
def my_bisection(f, a, b, tol):
    if np.sign(f(a)) == np.sign(f(b)): #check sign
        raise Exception("The scalars a and b do not bound a root")
    m = (a + b) / 2 #mid point
    if np.abs(f(m)) < tol:
        return m
    elif np.sign(f(a)) == np.sign(f(m)):
        return my_bisection(f, m, b, tol)
    else:
        return my_bisection(f, a, m, tol)
# Define the function f(x) = cos(x) - x
f = lambda x: np.cos(x) - x
# Apply bisection method on interval [0, 1] with tolerance 0.0001
root = my_bisection(f, 0, 1, 1e-4)
# Display the result
print("Root =", root)
print("f(root) =", f(root))
OUTPUT:
Root = 0.7391357421875
f(root) = -8.470073137478717e-05
```



### Explaining Each Step:

1. Defines a function to implement the Bisection Method
2. Checks if the initial interval actually contains a root
3. Calculates the midpoint of the current interval
4. Checks if the midpoint is close enough to the root (i.e., function value near zero)
5. Decides which half of the interval still contains the root
6. Recursively calls the function on the new interval
7. Defines the actual function  $f(x)=\cos(x) - x$
8. Calls the bisection function with a specific interval and tolerance
9. Prints the approximate root and the value of the function at that root

## 5. Analyze the convergence behavior of Newton-Raphson and Bisection methods with examples.

### NEWTON-RAPHSON METHOD:

The Newton-Raphson method has **quadratic convergence**. This means that the number of correct decimal places in the approximation roughly **doubles** with each iteration, provided the initial guess is close to the root.

#### Convergence Behavior:

Fast (Quadratic Convergence) near the root.

Very efficient if:

- The function is smooth,
- The derivative is non-zero near the root,
- The initial guess is close to the actual root.

May fail to converge if:

- Poor initial guess
- Derivative is zero or very small
- Function is not differentiable

### Example:

Solve  $f(x) = x^2 - 2$

Let  $f(x) = x^2 - 2$ ,  $f'(x) = 2x$

Initial guess:  $x_0 = 1$

$x_1 = x_0 - f(x_0)/f'(x_0) = 1 - (1 - 2)/2 = 1.5$

$x_2 = 1.5 - (1.5^2 - 2)/(2 \times 1.5) \approx 1.4167$

Converges to  $\sqrt{2} \approx 1.4142$  in just a few iterations.

### NEWTON-RAPHSON IMPLEMENTATION

```
f = lambda x: x**2 - 2
f_prime = lambda x: 2*x
def my_newton(f, df, x0, tol):
    if abs(f(x0)) < tol:
        return x0
    else:
        return my_newton(f, df, x0 - f(x0)/df(x0), tol)
estimate = my_newton(f, f_prime, 1, 1e-6) # 1e-6 means 0.000001
print("estimate =", estimate)
OUTPUT:
estimate = 1.4142135623746899
```

### BISECTION METHOD

The Bisection method has **linear convergence**. This means the error in the approximation is reduced by a constant factor in each step. For Bisection, this factor is 2. The size of the interval containing the root is halved with every iteration.

### Convergence Behavior:

Slow (Linear Convergence) but guaranteed if the function changes sign in the interval.

Always converges as long as  $f(a) \cdot f(b) < 0$

More robust, but slower than Newton-Raphson

### Example:

Solve  $f(x) = x^2 - 2$  on  $[1, 2]$

Initial interval:  $[1, 2]$

Midpoint  $m_1 = 1.5$ ,  $f(1.5) > 0$

Next interval:  $[1, 1.5]$

Then:  $1.25 \rightarrow 1.375 \rightarrow 1.4375 \rightarrow \dots$

Slowly approaches  $\sqrt{2} \approx 1.4142$

## BISECTION IMPLEMENTATION

```
import numpy as np

def my_bisection(f, a, b, tol):
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception(
            "The scalars a and b do not bound a root")
    m = (a + b) / 2
    if np.abs(f(m)) < tol:
        return m
    elif np.sign(f(a)) == np.sign(f(m)):
        return my_bisection(f, m, b, tol)
    elif np.sign(f(b)) == np.sign(f(m)):
        return my_bisection(f, a, m, tol)

f = lambda x: x**2 - 2
r1 = my_bisection(f, 1, 2, 1e-6)# 1e-6 means 0.000001
print("r1 =", r1)
print("f(r1) =", f(r1))
OUTPUT:
r1 = 1.4142136573791504
f(r1) = 2.687177129701013e-07
```

## Conclusion

Use Newton-Raphson for fast results when a good initial guess and derivative are available.

Use Bisection when safety and guaranteed convergence are more important, even if it's slower.